

Object Oriented Programming (Class Relationships)

Abbas Khalid

December 6, 2025

Table of contents

1 Communication Between Classes

2 Relationships Between Classes

Using a Non-member Function

- A class A can use the features of another class B through a global (non-member) function if that function takes an object of type B (or a reference/pointer to B) as a parameter.
- Inside the global function, only the public members of class B can be accessed, unless the function is declared as a friend of class B.

Example 1

Cash: 1000

```
1  class Ahmed;
2  double agent(Ahmed);
3  class Ahmed{
4      double cash;
5  public:
6      Ahmed(double c){cash = c;}
7      double getCash(){return cash;}
8  };
9  class Ali{
10     double cash;
11  public:
12     void displayCash(){cout<<"Cash: "<<cash<<endl;}
13     void cashInfo(Ahmed A){cash = agent(A);}
14 };
15 double agent(Ahmed A){return A.getCash();}
16 int main(){
17     Ali ali; Ahmed ahmed (1000);
18     ali.cashInfo(ahmed);    ali.displayCash();
19     return 0;
20 }
```

Example 2

Cash: 1000

```
1  class Ahmed;  
2  double agent(Ahmed*);  
3  class Ahmed{  
4      double cash;  
5  public:  
6      Ahmed(double c){cash = c;}  
7      double getCash(){return cash;}  
8  };  
9  class Ali{  
10     double cash;  
11  public:  
12     void displayCash(){cout<<"Cash: "<<cash<<endl;}  
13     void cashInfo(Ahmed* A){cash = agent(A);}  
14 };  
15 double agent(Ahmed* A){return A->getCash();}  
16 int main(){  
17     Ali ali; Ahmed ahmed (1000);  
18     ali.cashInfo(&ahmed);    ali.displayCash();  
19     return 0;  
20 }
```

Example 3

Cash: 1000

```
1 class Ahmed;
2 class Ahmed{
3     double cash;
4 public:
5     Ahmed(double c){cash = c;}
6     friend double agent(Ahmed);
7 };
8 class Ali{
9     double cash;
10 public:
11     void displayCash(){cout<<"Cash: "<<cash<<endl;}
12     void cashInfo(Ahmed A){cash = agent(A);}
13 };
14 double agent(Ahmed A){return A.cash;}
15 int main(){
16     Ali ali; Ahmed ahmed (1000);
17     ali.cashInfo(ahmed);    ali.displayCash();
18     return 0;
19 }
```

Using a Member Function

- A class A can use the features of another class B through one of its member functions if that function receives an object of type B (or a reference/pointer to B) as a parameter.
- Inside that member function, only the public members of class B can be accessed.

Example

```
1 class Ahmed{
2     double cash;
3     public:
4     Ahmed(double c){cash = c;}
5     double getCash(){return cash;}
6 };
7
8 class Ali{
9     double cash;
10    public:
11    void displayCash(){cout<<"Cash: "<<cash<<endl;}
12    void cashInfo(Ahmed A){cash = A.getCash();}
13 };
14 int main(){
15     Ali ali; Ahmed ahmed (1000);
16     ali.cashInfo(ahmed);    ali.displayCash();
17     return 0;
18 }
```

Cash: 1000

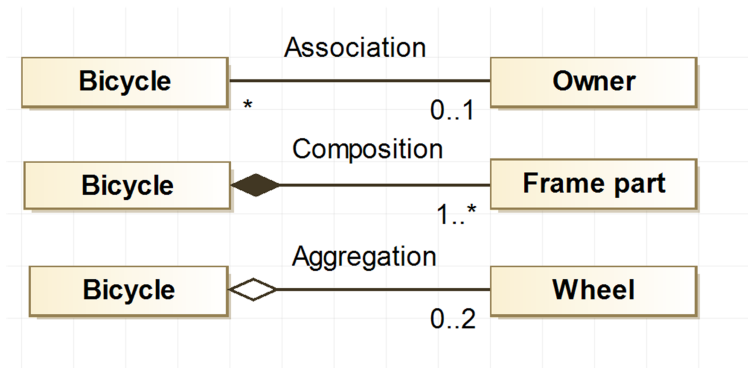
Relationships Between Classes

- Classes do not exist in isolation - they interact with each other.
- Common types of relationships:
 - 1 Association
 - 2 Aggregation
 - 3 Composition
 - 4 Inheritance
- The relationship between two classes depends entirely on the scenario: the same classes can represent association, aggregation, or composition based on how we design their interaction and whether one object owns or only uses the other.

UML Representation

- Association can be represented by a line between these classes with an arrow indicating the navigation direction. In case an arrow is on both sides, the association is known as a **bidirectional association**.
- There may be **one-to-one**, **one-to-many**, **many-to-one**, and **many-to-many** associations between objects.
- An aggregation is denoted as a solid line with an unfilled diamond at the end. The diamond is connected to the class that represents the aggregate (whole). Aggregation relationships may be bidirectional.
- A composition is denoted as a solid line with an filled diamond at the end. The diamond is connected to the class that represents the composite (whole). In a composition relationship, data usually flows in only one direction (i.e., from the whole class to the part class).

Pictorial Representation



Example 1: Car - Driver

- **Association** (Temporary use): A driver borrows a car to go to the market.
 - ① Both are created independently.
 - ② The car does NOT belong to the driver.
 - ③ The connection lasts only for that short trip.
 - ④ After the ride, the driver returns the car.
- **Aggregation** (Together but independent): A rent-a-car company **has many** cars and many drivers.
 - ① Both are created independently.
 - ② Any driver can be assigned any car.
 - ③ Cars and drivers can exist independently if separated.
 - ④ A driver may change cars; a car may be driven by different drivers.
- **Composition** (Part cannot exist without whole): A toy car **has a** driver figure permanently fixed inside it (glued or molded together).
 - ① Both are created at the same time.
 - ② The driver figure cannot be removed.
 - ③ If the toy car breaks, the driver figure also breaks.
 - ④ The driver's existence depends entirely on the toy car.

Example 2: Teacher - Student

- **Association** (Temporary use): A student stops a teacher in the hallway to ask a question.
 - ① Both are created independently.
 - ② They interact only for a moment.
 - ③ After the question, they go their separate ways.
 - ④ No long-term connection is formed.
- **Aggregation** (Together but independent): A teacher **has a** classroom of students.
 - ① Both are created independently.
 - ② Students can change teachers next year.
 - ③ Teachers can teach different groups each year.
 - ④ Both teacher and students exist independently if separated.
- **Composition** (Part cannot exist without whole): A teacher **has a** student record inside their personal gradebook.
 - ① The “student record” exists only inside that gradebook.
 - ② If the gradebook is lost or destroyed, the record is gone.
 - ③ The record cannot exist without the gradebook.
 - ④ A student cannot “die” if the teacher disappears, Student - Teacher composition is impossible. This is gradebook - student record composition.

Example 3: Employee - Company

A large company operates with many employees working in different departments, and even when some employees resign, the company continues its regular operations. One morning, a job applicant comes to the company for an interview and briefly meets an employee to ask a few questions. He was told that the HR department issues a work ID card for each newly hired employee. These ID cards remain valid only as long as the employee works in the company, and once the employee leaves, the card becomes useless and is immediately destroyed.

1 Which statement of the scenario represents Association?

The brief meeting between the job applicant and the employee during the interview represents association. (They interact temporarily, and neither owns the other.)

2 Which statement of the scenario represents Aggregation?

The company having many employees, and continuing to operate even when employees leave, represents aggregation. (The company has employees, but they are not owned; they can exist independently.)

3 Which statement of the scenario represents Composition?

The company creating a work ID card for each employee, and destroying the card when the employee leaves, represents composition. (The ID card cannot exist without its employee; its lifetime fully depends on the employee.)

Example 4: Doctor - Hospital

A doctor sometimes consults a nurse during surgeries, and this interaction happens only when needed. The hospital maintains a list of visiting surgeons; they work independently and are not owned by the hospital. Every patient in the hospital has a medical file that is created for them and is deleted when the patient leaves the hospital.

1 Which statement of the scenario represents Association?

A doctor sometimes consults a nurse during surgeries, and this interaction happens only when needed. (Temporary interaction, no ownership, both have independent lifetimes.)

2 Which statement of the scenario represents Aggregation?

The hospital maintains a list of visiting surgeons; they work independently and are not owned by the hospital. (Hospital has a collection but does not own surgeons; they exist independently.)

3 Which statement of the scenario represents Composition?

Every patient in the hospital has a medical file that is created for them and is deleted when the patient leaves the hospital. (File cannot exist without the patient, strong ownership, same lifecycle.)

Association

- It represents a general connection between two classes that allows them to communicate or interact when needed.
- It describes a broad “has-a” or “uses-a” relationship in which one class simply makes use of another’s functionality.
- It is considered the weakest type of relationship because it does not imply any structural dependency between the classes.
- It involves no ownership, meaning neither class controls the creation or destruction of the other.
- It allows objects to interact only temporarily, without forming any long-term or structural association.
- The lifetimes of the involved objects remain completely independent, as one object does not rely on the other for its existence. Both classes create their own objects independently in `main()` and their constructors run when they are created. No class creates or destroys the other’s object. Destructors execute in reverse order of creation.

Examples

① Teacher - Student

A Teacher teaches a Student. Both can exist without each other. Student and Teacher are created independently.

② Doctor - Patient

A doctor treats many patients. Patients can visit many doctors. No lifetime dependency.

③ Driver - Car

Driver uses the car, but neither owns each other. The car exists without the driver and the driver exists without the car.

Example 1

Teacher teaching...Students.

```
1 class Student {
2     public:
3         void attend(){cout<<"Students.\n";}
4 };
5
6 class Teacher {
7     public:
8         void teach(Student &s) {           // Teacher teaches Student
9             cout<<"Teacher teaching...\n";
10            s.attend();
11        }
12 };
13
14 int main() {
15
16     Student s; Teacher t;
17     t.teach(s);    // Association: temporary usage
18
19     return 0;
20 }
```

Example 2

```
1 class Patient {
2     int ID;
3     public:
4         Patient(int id):ID(id) {}
5         void getCheckup(){
6             cout<<"Patient " <<ID<<" is getting a checkup.\n";
7         }
8 };
9 class Doctor {
10     string Name;
11     public:
12         Doctor(string name):Name(name) {}
13         void examine(Patient &p) {
14             cout<<Name<<" is examining the patient.\n";
15             p.getCheckup();
16         }
17 };
18 int main() {
19     Doctor d("Dr. Rayhaab"); Patient p(101); d.examine(p);
20     return 0;
21 }
```

Dr. Rayhaab is examining the patient.
Patient 101 is getting a checkup.

Example 3

```
1 class Car {
2     string model;
3 public:
4     Car(string m):model(m) {}
5     void start(){cout<<model<<" is starting.\n";}
6 };
7
8 class Driver {
9     string name;
10 public:
11     Driver(string n):name(n) {}
12     void drive(Car &c) {
13         cout<<name<<" is driving the car.\n";
14         c.start();
15     }
16 };
17
18 int main() {
19     Driver d("Zakriya"); Car c("Honda Civic"); d.drive(c);
20     return 0;
21 }
```

Zakriya is driving the car.
Honda Civic is starting.

Aggregation

- It represents a whole - part relationship in which one class contains or uses another class without owning it.
- It describes a “has-a” relationship where the whole uses the part, but the part can still exist independently.
- It is a stronger relationship than simple association because it shows a structural link, but still without full ownership.
- It does not involve ownership, meaning the whole does not control the part's creation or destruction.
- It allows objects to interact in an ongoing way, but each object retains its own independent existence.
- The part object is created first and then supplied to the whole object during its creation. The whole stores a pointer or reference to the existing part. No copy constructor is called.
- The part is created independently, usually before the whole is constructed, and its constructor runs when it is created. The whole can use this existing part, but it does not control its lifetime. When the whole is destroyed, the part remains intact and is not automatically destroyed.

Examples

1 Car - Engine

A car has an engine but the engine is created outside the car and can exist without it. If car is destroyed, the engine still exists.

2 Library - Books

Books can exist even if the library building is destroyed. No lifetime dependency.

3 Team - Players

Players belong to a team, but they exist even if the team disbands.

Example 1

```
1 class Engine {
2     public:
3         Engine(){cout<<"Engine created.\n"; }
4         ~Engine(){cout<<"Engine destroyed.\n"; }
5 };
6 class Car {
7     Engine* eng;    // Car does NOT own engine: aggregation
8     public:
9         Car(Engine* e){eng = e; cout<<"Car created.\n";}
10        //This simply copies the pointer value. NO constructor of Engine is called.
11        //Nothing is being constructed here. The Engine object is already created in main.
12        ~Car(){cout<<"Car destroyed.\n";}
13 };
14 int main() {
15     Engine e; //Engine created independently
16     {
17         Car c(&e); //Car only uses the engine
18     } //Car dies here but Engine remains
19     cout<<"Engine alive. Car destroyed.\n";
20     return 0;
21 }
```

Engine created.
Car created.
Car destroyed.
Engine alive. Car destroyed.
Engine destroyed.

Example 2

```
1 class Book {
2     string title;
3 public:
4     Book(string t):title(t){cout<<title<<" created.\n";}
5     ~Book(){cout<<title<<" destroyed.\n";}
6     string getTitle(){return title;}
7 };
8 class Library {
9     string Name;
10    Book* b;
11 public:
12    Library(string name, Book* book):Name(name), b(book)
13        {cout<<Name<<"received a book:"<<b->getTitle()<<endl;}
14    ~Library() {cout<<Name<<"closed (book exists).\n";}
15 };
16 int main() {
17     Book bk("Learn C++");
18     {Library lib("Punjab Library", &bk); }
19     cout<<"Book still exists after library destruction.\n";
20 return 0;
21 }
```

Learn C++ created.
Punjab Library received a book: Learn C++
Punjab Library closed (book exists).
Book still exists after library destruction.
Learn C++ destroyed.

Example 3

```
1 class Player {
2     string name;
3 public:
4     Player(string n):name(n){cout<<name<<"created.\n";}
5     ~Player(){cout<<name<<"destroyed.\n";}
6     string getName(){ return name; }
7 };
8 class Team {
9     string team;
10    Player* p;
11 public:
12    Team(string team, Player* n):team(team), p(n)
13    {cout<<team<<" has player "<<p->getName()<<endl;}
14    ~Team(){cout<<team<<"disbanded(player exists).\n";}
15 };
16 int main() {
17     Player pl("Javed");
18     {Team t("HBL", &pl);}
19     cout<<"Player still alive after team is gone.\n";
20 return 0;
21 }
```

Javed created.
HBL has player Javed
HBL disbanded(player still exists).
Player still alive after team is gone.
Javed destroyed.

Composition

- It represents a strong whole - part relationship in which one class fully owns and manages the lifecycle of another class.
- It describes a strict “has-a” relationship where the part cannot logically or physically exist without the whole.
- It is the strongest form of class relationship because the whole is responsible for creating and destroying the part.
- It involves complete ownership, meaning the whole exclusively controls the part's existence.
- It requires the part to be created as a component inside the whole, making the relationship permanent and structured.
- The lifetime of the part is fully dependent on the whole, so when the whole is destroyed, the part is automatically destroyed as well. The part is automatically created inside the whole when the whole is constructed, and its constructor runs first. Its initialization is completed before the body of the whole's constructor executes. When the whole is destroyed, the part is automatically destroyed as well, with destruction occurring in reverse order.

Composition

- Member objects (parts) are constructed in the order in which they are declared (not in the order in which they are listed in the member *initializer – list*). They are constructed before their enclosing objects (whole).
- If a class member is an object of another class, making that member object *public* does not violate the encapsulation and hiding of that member object's *private* members. But, it does violate the encapsulation and hiding of the containing class's implementation, so member objects of class types should still be *private*, like all other data members.
- If a member object is not initialized through a member initializer, the member object's default constructor will be called implicitly. Values, if any, established by the default constructor can be overridden by *set* functions.
- We initialize member objects explicitly through member initializers. This eliminates the overhead of doubly initializing member objects - once when the member object's default constructor is called and again when *set* functions are called in the constructor body (or later) to initialize the member object.
- A compilation error occurs if a member object is not initialized with a member initializer and the member object's class does not provide a default constructor (i.e. the member object's class defines one or more constructors, but none is a default constructor).

Examples

① House - Room

Rooms do not exist independently of the House. If house is destroyed, rooms are destroyed automatically.

② Human - Heart

Heart cannot exist without the human body.

③ University - Departments

Department exists only as long as the university exists.

Example 1

```
1 class Room {
2     public:
3         Room(){cout<<"Room created.\n";}
4         ~Room(){cout<<"Room destroyed.\n";}
5 };
6
7 class House {
8     Room r;           //House owns Room: composition
9     public:
10        House(){cout<<"House created.\n";}
11        ~House(){cout<<"House destroyed.\n";}
12 };
13
14 int main() {
15     {
16         House h;      // House and its Room both created
17     }                // Both House and Room destroyed
18     cout<<"This is Composition.\n";
19     return 0;
20 }
```

Room created.
House created.
House destroyed.
Room destroyed.
This is Composition.

Example 2

```
1 class Heart {
2     int rate;
3     public:
4         Heart(int r = 72):rate(r)
5             {cout<<"Heart created with rate"<<rate<<"\n";}
6         ~Heart(){cout<<"Heart destroyed.\n";}
7 };
8
9 class Human {
10     string name;
11     Heart h;
12     public:
13         Human(string n):name(n), h(72)
14             {cout<<"Human " <<name<<" created.\n";}
15         ~Human(){cout<<"Human " <<name<<" destroyed.\n";}
16 };
17
18 int main() {
19     { Human person("ABC"); }
20     return 0;
21 }
```

Heart created with rate 72.
Human ABC created.
Human ABC destroyed.
Heart destroyed.

Example 3

```
1 class Department {
2     int deptID;
3 public:
4     Department(int id):deptID(id)
5         {cout<<"Department " <<deptID<<" created.\n";}
6     ~Department(){cout<<"Department destroyed.\n";}
7 };
8 class University {
9     string name;
10    Department dept;
11 public:
12    University(string n, int Id):name(n), dept(Id)
13        {cout<<"University" <<name<<" created.\n";}
14    ~University()
15        {cout<<"University" <<name<<" destroyed.\n";}
16 };
17 int main() {
18     {University u("ABC", 101);}
19 return 0;
20 }
```

Department 101 created.
University ABC created.
University ABC destroyed.
Department destroyed.

Example 4

```
1 class Date{
2     int day,month,year;
3     public:
4         Date(int d, int m, int y){day=d; month=m; year=y;}
5         void print(){cout<<day<<','<<month<<','<<year<<endl;}
6 };
7 class Student{
8     string name;
9     Date birthDate;
10    public:
11        Student(string n, Date d): birthDate(d){name = n;}
12        //birthDate(d) calls the default copy constructor of the Date class
13        void print(){
14            cout<<" Student Name = "<<name<<endl;
15            cout<<" Birth Date = "; birthDate.print();
16        }
17 };
18 int main(){
19     Date bd(4,2,2006); Student s("Muslim", bd); s.print();
20     return 0;
21 }
```

Student Name = Muslim
Birth Date = 4/2/2006

Example 5

```
1 class Time{
2     int hour,minute;
3     public:
4         Time(int h, int m){hour = h; minute = m;}
5         void print(){cout<<hour<<':'<<minute<<endl;}
6 };
7 class Exam{
8     string name;
9     Time startTime, endTime;
10    public:
11        Exam(string n, int sHr, int sMin, int eHr, int eMin):
12            startTime(sHr,sMin), endTime(eHr,eMin){name = n;}
13        void print(){
14            cout<<"Exam = "<<name<<endl;
15            cout<<"Starting Time = "; startTime.print();
16            cout<<"Ending Time = "; endTime.print();
17        }
18 };
19 int main(){
20     Exam e("OOP",10, 30, 12, 30); e.print();
21     return 0;
22 }
```

Exam = OOP
Starting Time = 10:30
Ending Time = 12:30

Conclusions

In this lecture we discussed about

- Association
- Aggregation
- Composition

In the next lecture we will discuss about

- Inheritance